Tehniline dokumentatsioon: Kaamera- ja objektituvastussüsteem

A. Luik

1. Grupi liikmed ja repositoorium

Nimi	Roll
A. Luik	Arendaja

Repositooriumid: - https://gitlab.cs.ttu.ee/alluik/ntr0670 - https://github.com/alekslu/ntr0670/

Kogu lähtekood, konfiguratsioon ja väljundfailid asuvad Git-repoos ülaltoodud aadressil.

2. Süsteemi arhitektuur

2.1 Üldine ülesehitus

```
WSL2 Ubuntu                          Windows
----------------------              ----------------------
capture_and_detect.py  ---------->  camera_win.py
(orkestreerimisloogika)             (USB kaamera capture)

detect_yolo.py         <----------  output/images/*.png
(YOLO inferents, CPU)               (ühiskausta kirjutus)
----------------------              ----------------------
```

WSL2 ei suuda USB-kaamerat stabiilselt avada, seetõttu käsitleb Windowsi-poolne skript kaamera ligipääsu. Mõlemad pooled kirjutavad failid Git-repo `output/` kausta, mida WSL ja Windows näevad läbi ühiseks monteeritud tee.

2.2 Andmevoog

1. Kasutaja käivitab WSL terminalist: `python wsl_scripts/capture_and_detect.py`
2. WSL skript teisendab failiteede formaadi WSL → Windows (`wslpath -w`)
3. WSL käivitab Windowsi Pythoni: `python.exe windows_scripts/camera_win.py --output <tee>`
4. Windows avab kaamera, soojendab kaadrit lühidalt, salvestab pildi `output/images/image_PPKKAAAA_tt_mm_`
5. WSL loeb pildi(d) `output/images/` kaustast
6. YOLOv8l mudel teeb inferentsi CPU-l
7. Tulemused salvestatakse:
   - `output/analysis/detections_all.json` — koondraport
   - `output/analysis/annotated/<nimi>_annotated.png` — märgendatud pilt

2.3 Failistruktuur

```
camera_proj/
  windows_scripts/
      camera_win.py          # Windowsi kaamera capture
  wsl_scripts/
      capture_wsl.py         # WSL käivitaja ja teekonnasild
      capture_and_detect.py  # Täisliini käivitaja (capture + detect)
      detect_yolo.py         # YOLO inferents ja tulemuste salvestus
      yolov8l.pt             # Mudeli kaalude fail (87 MB)
  output/
      images/                # Kaamerast salvestatud pildid
      analysis/
          detections_all.json
          annotated/         # Märgendatud pildid
  docs/
      spec.md
      report.md              # See dokument
  requirements-wsl.txt
  README.md
```

3. Kasutatav mudel

3.1 YOLOv8l

Parameeter	Väärtus
Mudel	YOLOv8l (large)
Käigu failinime	`yolov8l.pt`
Failisuurus	~87 MB
Treeningatlas	COCO (80 klassi)
Inferents	CPU (`device="cpu"`)
Sisendi resolutsioon	YOLOv8 vaikimisi (640 px)

Ultralytics YOLOv8 perekonnast on vaikimisi kasutatud kõige suurem ühe-etapiline detektor `yolov8l`, mis annab parima täpsuse CPU-peal. Mudeli fail laaditakse esimesel käivituskorral automaatselt alla.

Alternatiivsed mudelid:

Mudel	Failisuurus	Täpsus	Kiirus CPU-l
`yolov8n.pt`	6 MB	madalaim	kiireim
`yolov8s.pt`	22 MB	parem	hea
`yolov8m.pt`	52 MB	hea	aeglasem
`yolov8l.pt`	87 MB	väga hea	aeglane

3.2 Kasutatud COCO klassid

Koodis filtreeritakse YOLO 80 klassist välja ainult tuvastamiseks vajalikud:

YOLO klass	Kasutus süsteemis	Konfidentsilävi
`person`	isiku tuvastus	>= 0.50
`sports ball`	palli tuvastus	>= 0.25

Kõik ülejäänud 78 COCO klassi ignoreeritakse, et vältida valepositiivseid.

3.3 Pea paiknemise hinnang (head_estimate)

Puudub eraldi peade tuvastamise mudel. `head_estimate` bounding box arvutatakse iga tuvastatud isiku bounding boxist:

```
head_y2 = person_y1 + 0.35 × (person_y2 - person_y1)
head_box = [person_x1, person_y1, person_x2, head_y2]
```

See tähendab, et sinise kastiga tähistatakse iga isiku bounding box ülemine 35% kõrgusest — ligikaudne peapiirkond.

4. Tulemused päris andmetega

4.1 Koondstatistika

Tuvastus käivitati 7 pildil (`output/images/`). Mudel: `yolov8l.pt`, andmed: `output/analysis/detections_all.`

Näitaja	Väärtus
Töödeldud pilte	7 / 7
Tuvastatud isikuid kokku	35
head_estimate hinnanguid	35
Tuvastatud palle kokku	14
Pildid, kus pall tuvastati	6 / 7

4.2 Märgenduspiltide legend

Annotated piltidel on kasutatud kolme värvi:

Kast	Värv	Tähendus
Punane	(255, 80, 80)	Isik (`person`)
Roheline	(80, 220, 80)	Pall (`sports ball`)
Sinine	(80, 180, 255)	Pea hinnang (`head_estimate`, sees)

4.3 Näidispildid

Pilt 1 — testimage1 (5 isikut, 1 pall)

Figure 1: Märgendatud pilt testimage1: 5 isikut, 1 pall, konf kuni 0.94

Pilt 2 — testimage3

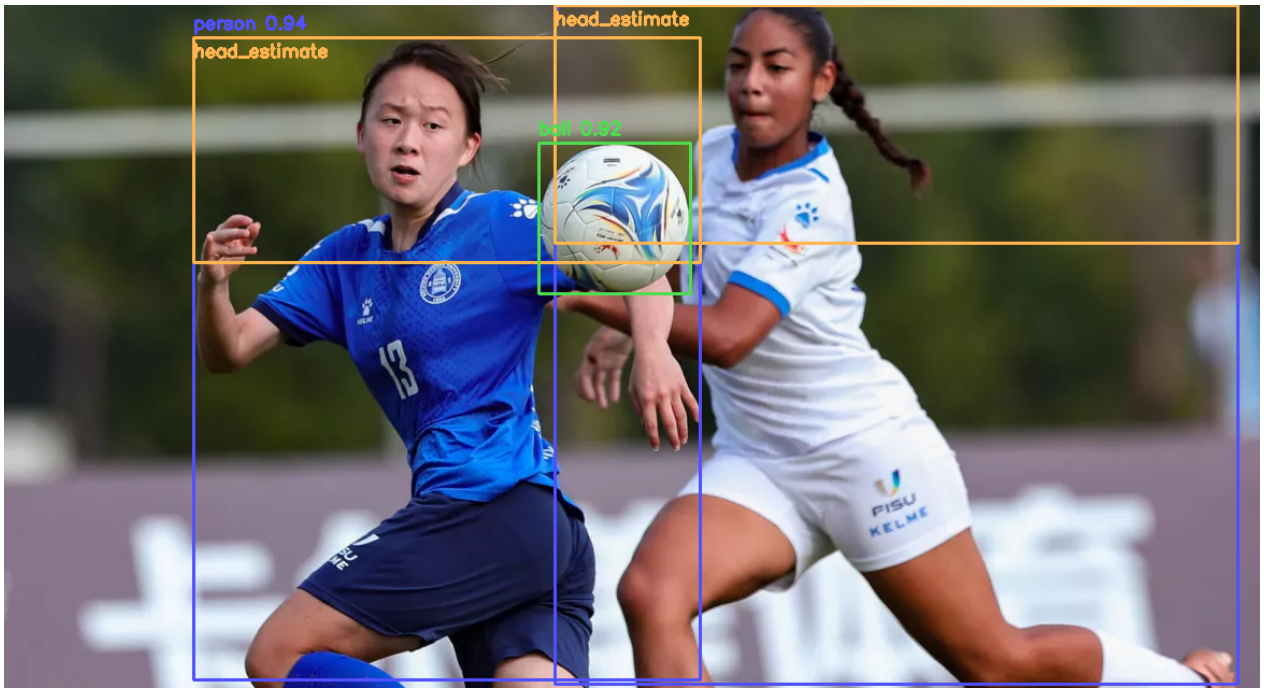

Figure 2: Märgendatud pilt testimage3

Pilt 3 — testimage5 (päris kaamerast salvestatud)

Tulemus: mudel ei tuvastanud sellelt kaadrist ühtegi isikut ega palli (`person=0, ball=0`). Põhjuseks on tõenäoliselt pildi madal kvaliteet, halb valgustus või nurk, mis erineb mudeli treeningandmetest.

Kuna YOLOv8l on treenitud COCO andmestikul üldiste stseenide jaoks, ei pruugi see võrkpallisaali spetsiifiliste tingimustega (taust, valgus, palli suurus kaugelt) hästi toime tulla. Tundlikkuse parandamiseks on kaks peamist võimalust:

- **Fine-tuning:** treenida YOLOv8 mudel uuesti spetsiifiliste võrkpallisaali andmetega, lisades eraldi klassi `volleyball`. See on kõige tõhusam viis tundlikkuse tõstmiseks antud stseeni jaoks.
- **Lisamudeli kasutamine:** kombineerida üldmudel spetsialiseeritud väikeste objektide detektoriga (nt SAHI — Slicing Aided Hyper Inference), mis jagab pildi väiksemateks tükkideks enne inferentsi ja parandab kaugel asuvate väikeste pallide tuvastust.

Figure 3: Märgendatud pilt testimage5_captured: päris kaamerakaadrilt, tuvastusi ei leitud

5. Käivitusjuhend

Eeldused

```
# WSL-i poolne keskkond
cd ~/ntr0670/camera_proj
python -m venv .venv
source .venv/bin/activate
pip install -r requirements-wsl.txt

# Windowsi poolne OpenCV
python.exe -m pip install opencv-python
```

Täisliini käivitus (capture + tuvastus)

```
python wsl_scripts/capture_and_detect.py
```

Ainult tuvastus olemasolevatel piltidel

```
python wsl_scripts/detect_yolo.py
```

Ühe pildi tuvastus

```
python wsl_scripts/detect_yolo.py --image output/images/image_15032026_18_42_07.png
```

Peamised argumendid

Argument	Vaikimisi	Kirjeldus
`--model`	`yolov8l.pt`	YOLO mudelfail
`--conf-person`	`0.50`	Isiku konfidentsilävi
`--conf-ball`	`0.25`	Palli konfidentsilävi
`--iou`	`0.45`	NMS IoU lävi — vt selgitust allpool
`--debug`	väljas	Prindi kõik toortuvastused

NMS IoU lävi selgitatud: YOLO mudel genereerib iga objekti jaoks mitu kattuvat bounding boxi. NMS (Non-Maximum Suppression) on järeltöötlusamm, mis neist üleliigsed eemaldab. IoU (Intersection over Union) mõõdab kahe kasti kattuvusastet vahemikus 0–1 (0 = ei kattu, 1 = täielik kattuvus).

Kui kahe kasti IoU on suurem kui seatud lävi (vaikimisi `0.45`), peetakse neid sama objekti tuvastusteks ja neist nõrgema konfidetnsiga kast eemaldatakse. Jääb alles vaid kõrgeima konfidentsiga kast.

- **Madal lävi (nt 0.3):** eemaldatakse ka vähem kattuvad kastid → vähem dubleerimist, aga võib kaotada lähedal asuvaid eri objekte
- **Kõrge lävi (nt 0.7):** lubatakse rohkem kattuvust → saab rohkem doubel-detektsioone, aga lähedased objektid eristatakse paremini

6. Dokumendi genereerimine

See kokkuvõte on kirjutatud Markdownis (`docs/report.md`) ja konverteeritud PDF-iks Pandoc abil. Dokumendi sisu on loodud Visual Studio Code'i GitHub Copilot agendi abiga.

Käsk PDF genereerimiseks (WSL terminalist repo juurkaustas):

```
cd ~/ntr0670/camera_proj
pandoc docs/report.md -o docs/report.pdf \
  --pdf-engine=xelatex \
  --resource-path=. \
  -V geometry:margin=2.5cm \
  -V fontsize=11pt
```

Vajalike tööriistade paigaldamine (Ubuntu/WSL):

```
sudo apt install pandoc texlive-xetex texlive-lang-european
```